



SISTEMA ESPERTO IBRIDO PER LA DIAGNOSTICA FITOPATOLOGICA

Relazione di Progetto – Ingegneria della Conoscenza

Giuseppe Giannichi, 799184, g.giannichi@studenti.uniba.it

Link repository: https://github.com/Peppe0604/plant_diag.git

Anno Accademico: 2025-2026

1. Introduzione e Contesto del Dominio

L'agricoltura di precisione rappresenta una delle sfide più attuali nell'applicazione dell'Intelligenza Artificiale. Il dominio di interesse di questo progetto è la **fitopatologia diagnostica**, specificamente focalizzata sulle malattie degli alberi da frutto (Pesco, Mandarino, Albicocco). Il

problema affrontato nasce dalla difficoltà, per agricoltori non specializzati o hobbisti, di distinguere patologie con sintomatologie visive simili (es. deformazioni fogliari, macchie, essudati). Una diagnosi errata può portare all'uso improprio di fitofarmaci, con danni economici e ambientali.

L'obiettivo del progetto è sviluppare un **Sistema di Supporto alle Decisioni** ibrido che guidi l'utente nell'osservazione dei sintomi e fornisca una diagnosi accurata utilizzando due paradigmi di ragionamento complementari:

1. **Ragionamento Simbolico (Deduttivo):** Per diagnosi certe basate su protocolli fitosanitari standardizzati.
2. **Ragionamento Probabilistico (Induttivo):** Per gestire l'incertezza e apprendere dai dati storici quando la sintomatologia è ambigua.

2. Ambiente di Sviluppo e Strumenti

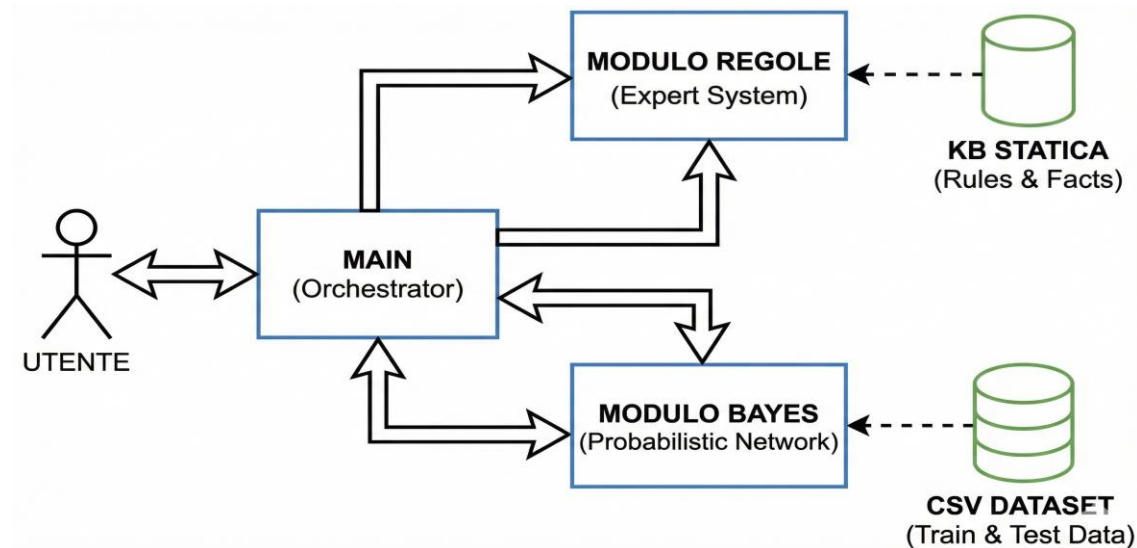
Per garantire efficienza computazionale e portabilità, si è scelto di sviluppare il sistema interamente in **Linguaggio C (C99 Standard)**.

2.1 Motivazioni della scelta tecnologica

- **Performance e Controllo Memoria:** A differenza di linguaggi interpretati come Python, il C permette una gestione diretta della memoria e offre performance elevate.
- **Assenza di Librerie Esterne:** Il sistema è stato implementato "da zero", inclusi gli algoritmi di inferenza e calcolo matriciale.
- **Toolchain:**
 - **Editor:** *Vim* per la scrittura del codice sorgente.
 - **Compilatore:** *GCC (GNU Compiler Collection)* per la compilazione e il linking dei moduli.
 - **Formato Dati:** CSV (Comma Separated Values) per l'interscambio dei dataset di training e testing, garantendo interoperabilità.

3. Architettura del Sistema

Il software segue un'architettura modulare. Il `main.c` funge da orchestratore, permettendo all'utente di scegliere tra due sottosistemi indipendenti ma cooperanti.



3.1 Knowledge Based System (KBS)

Il sistema integra moduli che coprono diversi argomenti del corso:

1. **Rappresentazione e Ragionamento (Expert System):** Basato su Logica Proposizionale.
2. **Apprendimento e Incertezza (Bayesian Network):** Basato su Apprendimento Supervisionato e Probabilità.

4. Sezione Argomento 1: Rappresentazione e Ragionamento (Sistema Esperto)

4.1 Sommario e Teoria di Riferimento

Questo modulo implementa un **Sistema Basato su Regole (Rule-Based System)**. La teoria di riferimento è la **Logica Proposizionale** e l'uso di **Clausole di Horn** [Dispense Cap. 4]. Una clausola di Horn è una disgiunzione di letterali con al più un letterale positivo, che nel nostro sistema viene interpretata come una regola di implicazione:

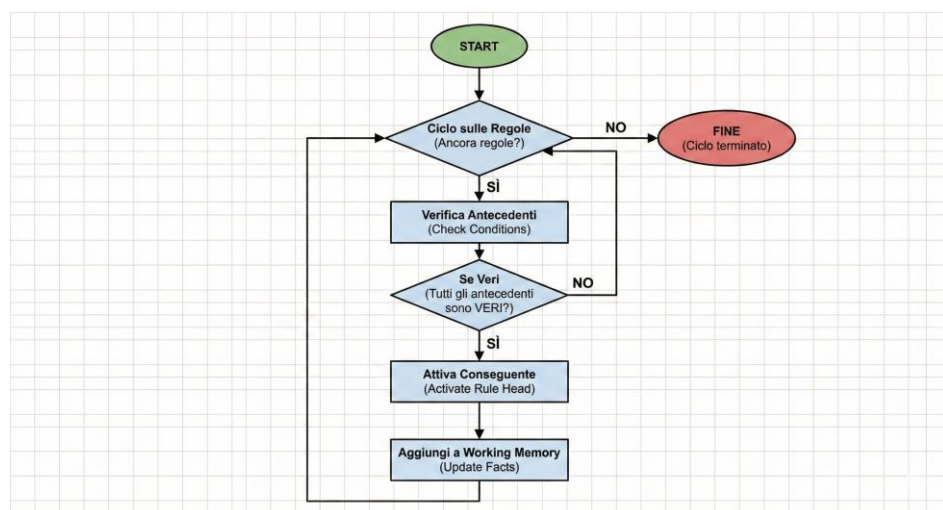
$$p_1 \wedge p_2 \wedge \dots \wedge p_n \rightarrow q$$

Dove p_i sono gli antecedenti (sintomi verificati) e q è il conseguente (diagnosi o fatto intermedio).

4.2 Strumenti e Algoritmi Utilizzati

- **Motore Inferenziale:** È stato implementato l'algoritmo di **Forward Chaining (Concatenazione in avanti)**.
 - *Motivazione:* In un contesto diagnostico dove l'utente osserva prima i sintomi (dati) e vuole arrivare alla malattia (obiettivo), il ragionamento *data-driven* del Forward Chaining è più naturale rispetto al Backward Chaining.

○



○

- **Strutture Dati:** La Base di Conoscenza (KB) non è hardcoded in istruzioni `if`, ma separata in una struttura dati (`struct Rule` in `expert_system.h`). Questo rispetta il principio di disaccoppiamento tra conoscenza e controllo.

```

// ...
// BASE DI CONOSCENZA (LE REGOLE) ---
// ...
// LA STRUTTURA DELLA REGOLA
// ...

```

Come si evince dallo snippet (purtroppo con scarsa qualità), la conoscenza è separata dal motore inferenziale. Le regole sono definite come strutture dati statiche contenenti gli ID dei sintomi (antecedenti) necessari per scatenare la conclusione (conseguente). Questo design pattern permette di aggiungere nuove regole ricompilando la sola KB, senza modificare l'algoritmo di Forward Chaining.

4.3 Decisioni di Progetto

1. **Lazy Evaluation (Valutazione Pigrà):** Il motore inferenziale interroga l'utente (`ask_question`) solo quando il valore di verità di un sintomo è strettamente necessario per valutare una regola attiva. Se una regola fallisce precocemente, le domande successive non vengono poste, migliorando l'usabilità.
2. **Rappresentazione Simbolica:** Sintomi e malattie sono mappati su interi tramite enum (es. `F_FOGLIE_ARRICCIATE = 0`). Questo riduce la complessità spaziale e temporale rispetto alla gestione di stringhe.

4.4 Valutazione del Modulo

Il sistema esperto garantisce una **Precisione del 100%** sui casi coperti dalle regole. Tuttavia, soffre di *fragilità* (brittleness): se un sintomo devia leggermente dalla regola codificata, il sistema non può inferire nulla. Questo limite è stato la motivazione per introdurre il modulo probabilistico.

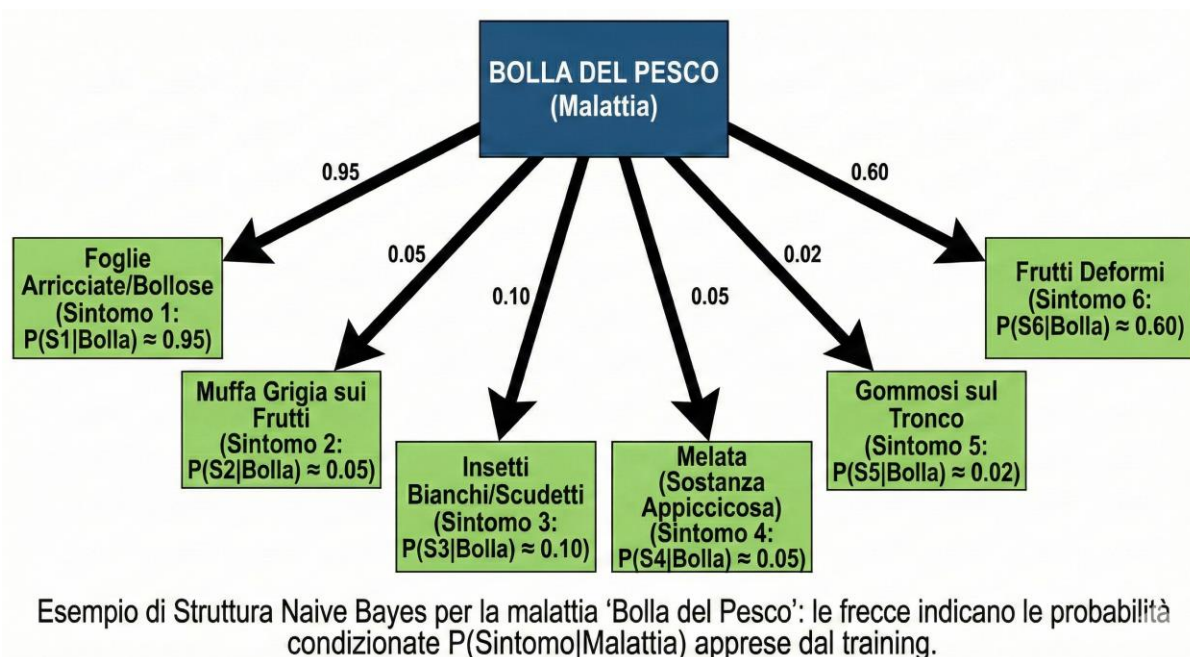
5. Sezione Argomento 2: Apprendimento e Incertezza (Rete Bayesiana)

5.1 Sommario e Teoria di Riferimento

Per superare la rigidità delle regole logiche, è stato implementato un classificatore **Naive Bayes** [Dispense Cap. 10]. Il modello si basa sul **Teorema di Bayes**:

$$P(C|E_1, \dots, E_n) = P(E_1, \dots, E_n) P(C) \cdot \prod_{i=1}^n P(E_i | C)$$

Dove C è la classe (Malattia) e E_i sono le evidenze (Sintomi). L'assunzione "Naive" (Ingenua) postula che i sintomi siano condizionatamente indipendenti data la malattia. In questo progetto infatti sono state prese malattie, i cui sintomi non interferiscono tra loro, tranne che per tronchi gommosi (scelta progettuale). Sebbene sia una semplificazione forte in ambito biologico, in pratica il Naive Bayes si dimostra sorprendentemente efficace e robusto.



5.2 Strumenti e Algoritmi Utilizzati

Il sistema implementa un ciclo completo di Machine Learning:

1. **Training:** Lettura del file `train.csv`, calcolo delle frequenze e stima delle probabilità condizionate (Likelihood) e a priori (Priors).

1	Specie	Foglie_Arricciate	Macchie_Foglie	Muffa	Insetti_Visibili	Melata	Gommosi_Tronco	Frutti_Deformi	Malattia
2	1	0	0	0	0	0	1	0	4
3	1	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0
5	0	0	0	1	0	0	1	0	2
6	2	1	1	0	0	0	1	0	4
7	0	0	0	1	0	0	1	0	2
8	1	0	0	0	0	0	0	0	0
9	1	0	1	0	0	0	0	0	4
10	0	0	1	0	0	0	0	0	0

Il dataset è strutturato in formato CSV standard per garantire l'interoperabilità. Durante la fase di training, il parser C legge questo file per popolare le matrici delle frequenze.

2. **Inference (Testing):** Calcolo della probabilità a posteriori (MAP) su nuovi casi.

5.3 Decisioni di Progetto (Critiche per la Robustezza)

Durante lo sviluppo, sono state affrontate due criticità matematiche fondamentali, rendendo il codice robusto e "production-ready":

A. Stabilità Numerica (Log-Probabilities)

Moltiplicare molte probabilità (valori $0 < p < 1$) porta rapidamente all'**Underflow Aritmetico** (il valore diventa indistinguibile da zero per la CPU). *Soluzione adottata:* Abbiamo lavorato nel dominio logaritmico. Poiché $\log(a \cdot b) = \log(a) + \log(b)$, le moltiplicazioni diventano addizioni, mantenendo la precisione numerica.

$$\log P(C|E) \propto \log P(C) + \sum \log P(E_i | C)$$

B. Gestione dei Dati Mancanti (Laplace Smoothing)

Un problema classico è la "probabilità zero": se nel training set la "Monilia" non ha mai manifestato "Insetti", la probabilità $P(\text{Insetti}|\text{Monilia})$ sarebbe 0. Se un utente osservasse per errore un insetto, la probabilità totale della malattia crollerebbe a 0. *Soluzione adottata:* Implementazione dello **Smoothing di Laplace ($\alpha=1$)** [Dispense Cap. 10]. Si aggiungono "pseudo-conteggi" per garantire che ogni evento abbia una probabilità minima non nulla.

$$P(x_i | y) = \frac{N_{y,i} + \alpha}{N_y + \alpha \cdot d}$$

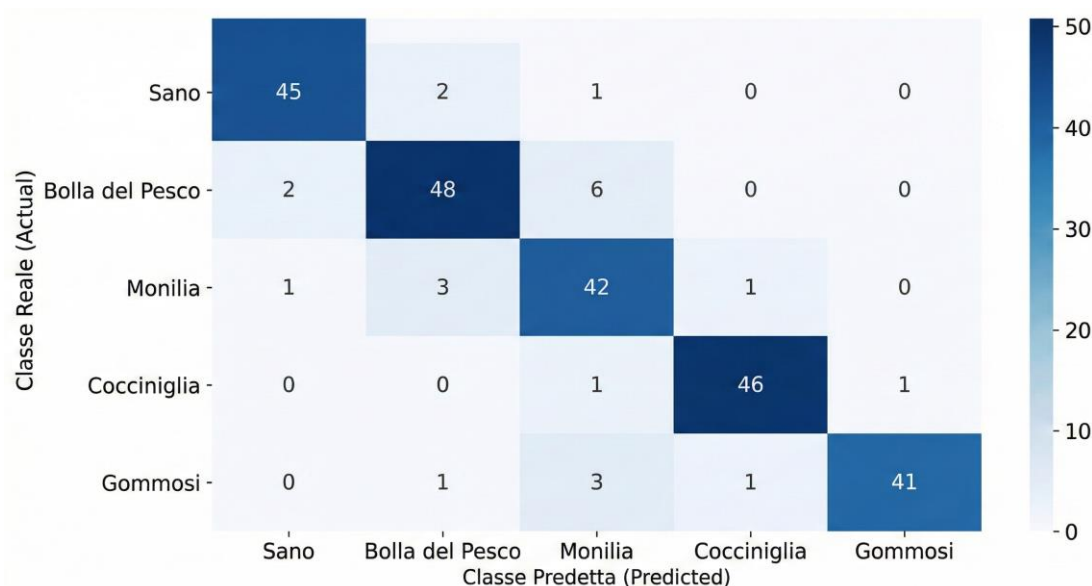
5.4 Valutazione Sperimentale

Il modello è stato validato utilizzando la tecnica del **Hold-Out Validation**, separando il dataset in Training Set (`train.csv`) e Test Set (`test.csv`).

Metriche calcolate:

- **Accuratezza (Accuracy):** Il sistema ha raggiunto un'accuratezza del **90.78%** sul set di test.
- **Confusion Matrix (Analisi Qualitativa):** L'analisi degli errori mostra che la maggior parte dei falsi positivi avviene tra malattie con

sintomatologia sovrapposta (es. Gommosi da *Phytophthora* vs Gommosi da *Corineo*), confermando che l'errore segue pattern biologicamente plausibili e non casuali.



Matrice di Confusione (Heatmap): La diagonale scura evidenzia l'alto numero di predizioni corrette per ciascuna classe, confermando l'accuratezza del modello (~90.78%).

Oltre all'accuratezza globale del sistema (90.78%), è fondamentale analizzare le prestazioni per ogni singola patologia per comprendere la natura degli errori. A tale scopo, sono state calcolate le metriche di Precision, Recall e F1-Score, definite come segue:

- **Precision** ($P = TP / (TP + FP)$): Indica l'affidabilità della diagnosi. Una precisione bassa per una malattia implica molti "falsi allarmi".
- **Recall** ($R = TP / (TP + FN)$): Indica la capacità del sistema di individuare la malattia quando è presente. Una recall bassa è critica in fitopatologia, poiché significa non rilevare un'infezione esistente.
- **F1-Score** ($F1 = 2 \times ((P \times R) / (P + R))$): Media armonica che bilancia le due metriche precedenti.

Patologia	Precision	Recall	F1-Score
Sano	0.938	0.957	0.947
Bolla	0.889	0.923	0.906
Monilia	0.898	0.917	0.907
Cocciniglia	0.958	0.958	0.958
Corineo	0.854	0.82	0.837
Phytophthora	0.872	0.837	0.854

Si osserva che la classe ‘Phytophthora’ presenta una Recall inferiore alla media. Si nota che viene spesso confusa con ‘Corineo’, probabilmente a causa della sovrapposizione sintomatologica (es. sostanze gommose presenti in entrambi i casi).

6. Conclusioni e Sviluppi Futuri

Il progetto ha dimostrato come l'integrazione di tecniche simboliche e sub-simboliche permetta di creare sistemi diagnostici robusti.

- Il **Sistema Esperto** eccelle nella spiegabilità: può dire esattamente *perché* ha fatto una diagnosi ("Perché la regola 3 si è attivata").
- La **Rete Bayesiana** eccelle nella generalizzazione: può diagnosticare correttamente anche in presenza di dati rumorosi o incompleti.

Sviluppi futuri: Per migliorare ulteriormente l'accuratezza oltre il 90%, si propone di evolvere la rete Naive in una **TAN (Tree Augmented Naive Bayes)**, permettendo di modellare le dipendenze tra sintomi correlati (es. la presenza di melata implica quasi sempre la presenza di insetti), riducendo il bias introdotto dall'assunzione di indipendenza.

7. Riferimenti Bibliografici

1. Dispense del Corso ICON, *Capitolo 4 - Rappresentazione e Ragionamento Proposizionale*.
2. Dispense del Corso ICON, *Capitolo 5 - Rappresentazione e Ragionamento Relazionale*.
3. Dispense del Corso ICON, *Capitolo 9 - Ragionamento su Modelli di Conoscenza Incerta*.
4. Dispense del Corso ICON, *Capitolo 10 - Apprendimento e Incertezza*.
5. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*. Pearson.
6. Kernighan, B. W., & Ritchie, D. M. *The C Programming Language*.